

Numpy (Numerical Python)

Numpy or Numerical Python is a foundational library in the python data ecosystem serving as high performance engine for libraries like Pandas and Matplotlib.

It provides an efficient multi-dimensional array object called ndarray and tools for working with these arrays through low level optimization

Standard Python vs. Numpy

- Memory Efficiency:- A standard python integer is a complex object that occupies significant memory typically 28 bytes for a single number like 1. In contrast, Numpy allows for precise bit controlled sizes.
- Storage:- Python lists store pointers to objects scattered in memory. Numpy arrays use contiguous memory storage.
- CPU Instructions:- Because of its contiguous nature and fixed data types, Numpy ~~array~~ can leverage advanced CPU instructions for vector and matrix operations.
- Performance:- Operations that take milliseconds in standard Python (using list comprehension) often take only microseconds in Numpy.

Installation

i) Install Numpy Library:-

`pip install numpy` (Noneed if working on Colab, etc)

ii) Import numpy:-

`import numpy as np`

iii) check version:-

`np.__version__`

Data Types and Bit Control

TYPE	BIT SIZES	DESCRIPTION
Integers	int8, int16, int32, int64	int8 uses 1 byte; int64 is the 8-byte default on 64 bit platforms
Floats	float16, float32, float64	float64 is the standard for high-precision decimals.

Example of Type Control

```
import numpy as np
```

```
# Creating an array with a specific 8-bit integer type
```

```
a = np.array([1, 2, 3], dtype=np.int8)
```

Create Arrays in NumPy

1) 1-D Array:-

arr = np.array([1, 2, 3, 4, 5])

(, represented by <class 'numpy.ndarray'>

use of type ()
print(type(arr))

o/p

2) 2-D Array:-

arr = np.array([[1, 2, 3], [4, 5, 6]])

o/p → $\begin{bmatrix} [1, 2, 3] \\ [4, 5, 6] \end{bmatrix}$

In Colab or Notebook
o/p will be only
numpy.ndarray

3) Higher Dimensional array

arr = np.array([1, 2, 3, 4], ndmin=5)

o/p → $\begin{bmatrix} \begin{bmatrix} \begin{bmatrix} \begin{bmatrix} \begin{bmatrix} 1, 2, 3, 4 \end{bmatrix} \end{bmatrix} \end{bmatrix} \end{bmatrix} \end{bmatrix}$
5-D array

4) Reshape:- create array with diff shape

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])

arr = arr.reshape(3, 3)

o/p → $\begin{bmatrix} [1, 2, 3] \\ [4, 5, 6] \\ [7, 8, 9] \end{bmatrix}$

Create 2D array
from 1D array

5) Flattening arrays

↳ convert multidimensional array to 1D array

we use reshape(-1) for flattening

```
a = np.array([1, 2, 3], [4, 5, 6])
```

```
arr = a.reshape(-1)
```

o/p → [1, 2, 3, 4, 5, 6]

6) Convert ndarray to list

↳ (convert "numpy.ndarray" to "list" datatype)

```
arr = np.array([1, 2, 3, 4, 5])
```

```
print(type(arr)) → o/p → <class "numpy.ndarray">
```

```
l = arr.tolist()
```

type(l) o/p → list

7) arange() function

↳ used to create sequence of numbers.
↳ generate "numpy.ndarray"

```
np.arange(10)
```

o/p → [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

np.arange(start, stop, step, dtype)

default 0

provided

default 1

data type
for array

'f' - float

'i' - int

Array creation using Numpy functions

1) zeros :- create array with all elements 0, with given dimensions.

$a = \text{np.zeros}(2, 3, \text{dtype} = \text{int})$

o/p $\rightarrow \begin{bmatrix} [0, 0, 0] \\ [0, 0, 0] \end{bmatrix}$

int or tuple of int for multi-dimensional array

$\text{np.zeros}(\text{shape} = (), \text{dtype} = \text{int}, \text{order} = ())$

$\{ 'C', 'F' \}$
row-major order (C-style) column-major order (Fortran style)

2) ones :- Array with all elements as 1 with dims

$a = \text{np.ones}(2, 3, \text{dtype} = \text{int})$

o/p $\rightarrow \begin{bmatrix} [1, 1, 1] \\ [1, 1, 1] \end{bmatrix}$

$\text{np.ones}(\text{shape}, \text{dtype}, \text{order})$

3. arange ()

4) linspace :- create a 1D array of linear space numbers/values, by default 50 linspace.

$a = \text{np.linspace}(0, 10, \text{num}=5)$

o/p $\rightarrow [0., 2.5, 5., 7.5, 10.]$

5 nums b/w 0 & 10

no of samples to generate

linspace(start, stop, num, endpoint, retstep, dtype)

if True, return
tuple - (samples, step)

spacing b/w
samples

Datatype
never int

bool
if True, stop
included otherwise
not. default
True

ex:-

$p = \text{np.linspace}(2, 5, 5, \text{endpoint}=\text{False}, \text{retstep}=\text{True})$

o/p $\rightarrow (\text{array}([2., 2.6, 3.2, 3.8, 4.4]), 0.6)$

array of 5 samples

step

5) eye :- Return 2D array with 1 at diagonal & 0 at elsewhere.

np.eye(N, M, K, dtype, order)

rows

columns

Move diagonal

+ve $\rightarrow$ move upward

-ve $\rightarrow$ move downward

$a = \text{np.eye}(4, 4, k=-1)$

4x4
 $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

for $k=0$

$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$

for $k=-1$
 1 diagonal
 down

`e = np.eye(2, 3)`

o/p $\rightarrow \begin{bmatrix} 1, 0, 0 \\ 0, 1, 0 \end{bmatrix}$

6) identity :- same as eye() but take only 1 arg.
 $\therefore \text{row} = \text{column}$ [Must be a square matrix]

`d = np.identity(3)`

o/p $\rightarrow \begin{bmatrix} 1, 0, 0 \\ 0, 1, 0 \\ 0, 0, 1 \end{bmatrix}$

} Generate square matrix with given int

`np.identity(n, dtype)`

row = column

dtype

7) fromiter :- create ndarray from iterable object,
return 1D ndarray object

`lst = [0, 2, 4, 6, 8]`

`it = iter(lst)`

`x = np.fromiter(it, dtype=float)`

o/p $\rightarrow [0, 2, 4, 6]$

↳ numpy.ndarray

Accessing Array Elements

① Iterating using nditer()

easy to iterate each scalar element in array, even for complex dimensions.

```
arr = np.array([[1,2,3], [4,5,6]])
```

```
for x in np.nditer(arr):  
    print(x)
```

o/p → $\begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{matrix}$ } access array like 1D

② nditer() with step :-

```
arr = np.array([[1,2,3,4], [5,6,7,8]])
```

```
for x in np.nditer(arr[::2]):  
    print(x)
```

o/p → $\begin{matrix} 1 \\ 3 \\ 5 \\ 7 \end{matrix}$

③ Iterate using ndenumerate()

return index of elements while iterating.

enumerate → mention sequence num one by one

```
arr = np.array([1,2,3], [4,5,6])
```

```
for idx, x in np.ndenumerate(arr):  
    print(idx, x)
```

o/p → $\begin{matrix} (0,0) 1 \\ (0,1) 2 \\ (0,2) 3 \\ (1,0) 4 \\ (1,1) 5 \\ (1,2) 6 \end{matrix}$

return tuple of index in multi-dim array.

Array Indexing

① 1D array indexing :- indexing starts from 0.

`arr = np.array([0, 1, 2, 3, 4])`

`arr[0]` ~~`arr`~~ $\Rightarrow 1$

② 2D array indexing :- provide comma-separated indices.

`a = np.array([[1, 2, 3], [4, 5, 6]])`

`a[0, 1]`, `a[0, 2]`, `arr[0][1]`
 $\downarrow$ $\downarrow$ $\downarrow$
 2 3 2

③ 3D array indexing :- 3 indices

`arr = [[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]]`

`arr[0, 1, 2]`, `arr[0][1][2]`

$\searrow$ 6 $\swarrow$

④ Negative indexing :- starts from the end.

`arr = np.array([[1, 2, 3], [4, 5, 6]])`

`arr[-1, -1]` $\Rightarrow 6$

Array Slicing

Access elements between start and end indices.

`arr[start:end]`

`arr[start:end:step]`

end index!
is not included

0	1	2	3	4	5	6	7	8
1	2	3	4	5	6	7	8	9
-9	-8	-7	-6	-5	-4	-3	-2	-1

← -ve

arr[1:5] $\Rightarrow$ [2, 3, 4, 5]
 not included

arr[4:] $\rightarrow$ when end not given $\Rightarrow$ default len(arr)
 $\downarrow$
 [5, 6, 7, 8, 9]

Q. Use 2D Array
 0 [1, 2, 3, 4, 5]
 1 [6, 7, 8, 9, 10]

arr[1, 1:4] $\Rightarrow$ [7, 8, 9]

arr[0:2, 2] $\Rightarrow$ [3, 8]

arr[0:2, 1:4] $\Rightarrow$ [[2, 3, 4, 5],
 [7, 8, 9, 10]]

Attributes of ndarray.

a = [42] b = [1, 2, 3, 4, 5] c = [[1, 2, 3],
 [4, 5, 6]]

d = [[1, 2, 3] [7, 8, 9]
 [4, 5, 6] [10, 11, 12]]

- ① ndim - return dimension of the array
- ② shape :- return shape - (row, col) of array.
- ③ size :- return total no of elements in array.

	a	b	c	d
ndim	0	1	2	3
shape	()	(5,)	(2, 3)	(2, 2, 3)
size	1	5	6	12

Other Numpy functions

① insert() :- Insert value at given place.

→ changes are not permanent need other variable

```
a = np.array([1, 2, 3, 4, 5])  
b = np.insert(a, 2, 100)
```

o/p → [1, 2, 100, 3, 4, 5]

② append() :- Insert value at array end.

```
a = [1, 2, 3, 4, 5]  
b = np.append(a, 200)
```

o/p → [1, 2, 3, 4, 5, 200]

③ ceil() :- yield to upper closest int for given float

```
a = [1.2, 5.8, 9.4]
```

```
a = np.ceil(a) ⇒ [2, 6, 10]
```

④ floor() :- yield closest lower int for float.

```
a = np.floor(a) ⇒ [1, 5, 9]
```

⑤ around() :- yield to closest int.

$\left. \begin{array}{l} \cdot 5 < \Rightarrow \text{lower} \\ \cdot 5 \geq \Rightarrow \text{upper} \end{array} \right\}$

```
np.around(a)
```

o/p → [1., 6., 9.]

⑥ argmax() :- return index of max element in array

(arr = np.array([10, 15, 2, 1, 8, 16]))

(np.argmax(arr))

o/p → 5 (Max element: 16)

⑦ argmin() :- return index of min element in array

(np.argmin(arr))

o/p → 3

⑧ size() :- Return size of an array.

(np.size(arr)) ⇒ 6

⑨ where() :- Return indices of values, for which given condⁿ is satisfied

(np.where(condition, True (Replace this value),
False (Replace this value)))

a = [1.2, 5.8, 9.4, 5.6, 12.5, 4.8]

a = np.where(a > 5)

o/p → (array([1, 2, 3, 4], dtype=int64),)

a = np.where(a > 5, 10, 0)

o/p → [0, 10, 10, 10, 10, 0]

Random Number Generation

• random module

* Built-in module for random number generation.

① rand() :-

* Generates an array of random floating-point numbers uniformly distributed over the interval [0.0, 1.0]

$a = \text{np.random.rand}(10)$ $b = \text{np.random.rand}(2,3)$
o/p $\rightarrow [0.44, \dots, 0.3046]$ o/p $\rightarrow \begin{bmatrix} 0.66 & 0.34 & 0.12 \\ \dots & \dots & \dots \end{bmatrix}$
1D array with size 10 2D array with size 2x3

② random() :- only diff in the argument.

$a = \text{np.random.random}(10)$ $b = \text{np.random.random}((2,3))$
o/p $\rightarrow [\dots, \dots, \dots]$ o/p $\rightarrow \begin{bmatrix} \dots & \dots & \dots \\ \dots & \dots & \dots \end{bmatrix}$
size is given in tuple.

③ randf() :-

similar to random
Tuple-Base Syntax for Multidimensional array.

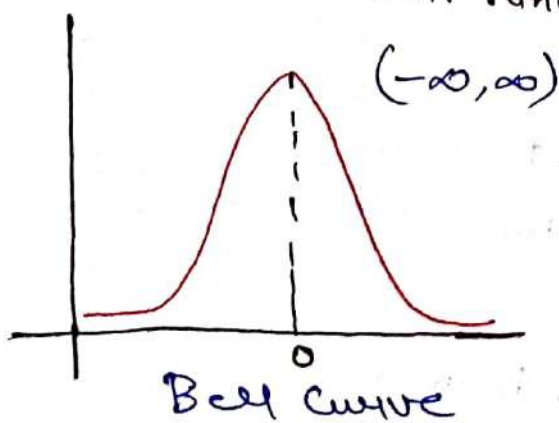
④ randint() :- random integer numbers.

$\text{np.random.randint}(\text{low}, \text{high}, \text{size}, \text{dtype} = \text{''})$
default = 1

⑤ randn() :-

* used to generate random numbers from a standard normal (Gaussian) distribution.

$a = \text{np.random.randn}()$



Array Slicing ^{exclusive}

$\text{arr}[\text{start} : \text{stop} : \text{step}]$

- $\text{arr}[1:4] \rightarrow$ indexes 1, 2, 3
- $\text{arr}[:3] \rightarrow$ first three elements
- $\text{arr}[2:] \rightarrow$ from index 2
- $\text{arr}[:, 2] \rightarrow$ every 2nd element
- $\text{arr}[:, :-1] \rightarrow$ reverse

2D/3D Slicing

for 2D:-

$\text{arr}[\text{row-slice}, \text{column-slice}]$

example:-

- $\text{arr}[:, :, 0] \rightarrow$ column 0 from all blocks
- $\text{arr}[:, 0, :] \rightarrow$ row 0 from all blocks
- $\text{arr}[0, :, :] \rightarrow$ entire block 0

% colon

$\rightarrow$ means select everything along that dimension

- $\text{arr}[:, 1] \rightarrow$ column 1 from all rows
- $\text{arr}[0, :] \rightarrow$ all columns of row 0.
- $\text{arr}[:, :, 1] \rightarrow$ column 1 from all blocks.

Step

$\text{arr}[\text{start} : \text{stop} : \text{step}]$

- $\text{arr}[:, :2] \rightarrow$ every second element
- $\text{arr}[:, :, :2] \rightarrow$ every second column
- $\text{arr}[:, :2, :] \rightarrow$ every 2nd row
- $\text{arr}[:, :, :, :2] \rightarrow$ every 2nd column in 3D.

for 3D:-

$\text{arr}[\text{block-slice}, \text{row-slice}, \text{column-slice}]$

Boolean Arrays (Masks in Numpy)

A boolean array, also called as mask, is an array containing only True and False values.

- It is mainly used to filter, select or modify elements based on condition.

Creating a boolean array

```
arr = np.array([10, 20, 30, 40, 50])
```

```
mask = arr > 30
```

```
print(mask)
```

O/P → [False, False, False, True, True]

or we can directly use

```
arr[arr > 30]
```

same output

Multiple Conditions

AND → & use & instead of and with Numpy arrays

```
arr[(arr > 20) & (arr < 50)] → [30, 40]
```

OR → | use | instead of or with Numpy arrays

```
arr[(arr > 20) | (arr < 50)] → [10, 50]
```

NOT → ~

```
arr[~(arr > 30)] → [10, 20, 30]
```

Masking with np.where()

```
arr = np.array([10, 20, 30, 40, 50])
```

```
result = np.where(arr > 30, 1, 0)
```

```
print(result)
```

O/P → [0 0 0 1 1]